

E.F. CODD'S 12 RULES

for a True Relational Database System

1. Who Was E.F. Codd & Why Do His Rules Matter?

Edgar Frank Codd (1923–2003) was a British computer scientist who worked at IBM. In 1970, he published his landmark paper 'A Relational Model of Data for Large Shared Data Banks' — which laid the entire foundation of modern relational databases.

By the 1980s, many software vendors were labelling their products as 'Relational DBMS' without actually following relational principles. To counter this, Codd published his 12 Rules in 1985 as a precise checklist. Any system claiming to be a true RDBMS must satisfy all 12 rules (plus Rule 0).

2. Rule 0 — The Foundation Rule (The Master Rule)

Rule 0 is the umbrella rule. All other rules depend on it.

Rule 0	The Foundation Rule
Definition	A relational DBMS must manage its data entirely through its relational capabilities alone. It cannot use any non-relational paths (like direct file access or pointer-based navigation) to read or write data.
Real World	MySQL, PostgreSQL, Oracle — users access all data ONLY through SQL (relational). There is no way to 'bypass' the relational engine and directly read a file.
Violation	A system that lets you bypass SQL and read raw disk files for performance — even partially — fails Rule 0 and is NOT a true RDBMS.
Remember	Rule 0 is the gateway — if a system fails Rule 0, the remaining 12 rules are irrelevant.

3. Codd's 12 Rules (Rule 1 to Rule 12)

Rule 1 — The Information Rule

Rule 1	The Information Rule
Definition	ALL information in a relational database must be represented in one and only one way: as values in columns within rows of tables. No pointers, no hidden links, no out-of-band storage.

Real World	A STUDENT table stores student name, roll number, and age as column values in rows. There are no pointer columns saying 'see record #27 in another file'.
Violation	A system that uses hidden row pointers (like early IMS hierarchical databases) to connect records instead of foreign keys violates Rule 1.
Remember	Every piece of data = a value in a cell of a table. Nothing else is allowed.

Rule 2 — Guaranteed Access Rule

Rule 2	Guaranteed Access Rule
Definition	Every single data item (atomic value) in a relational database must be logically accessible using a combination of: Table Name + Primary Key value + Column Name. There must be NO ambiguity.
Real World	SELECT Name FROM Student WHERE Roll_No = 101; — This uniquely identifies 'Alice' using table name (Student), primary key (Roll_No=101), and column name (Name).
Violation	A database where two rows can have the same primary key (duplicate rows allowed) — you can no longer guarantee which row you get. This violates Rule 2.
Remember	Table Name + Primary Key + Column Name = guaranteed to return exactly one value. This is why PRIMARY KEY must be UNIQUE and NOT NULL.

Rule 3 — Systematic Treatment of NULL Values

Rule 3	Systematic Treatment of NULL Values
Definition	NULL values must be supported uniformly by the DBMS for representing missing or inapplicable information, regardless of data type. NULL is distinct from zero, blank, or empty string.
Real World	A patient's date of death is NULL if they are alive. An employee's middle name is NULL if they have none. NULL means 'unknown' or 'not applicable' — not zero.
Violation	Storing 0 in a numeric field to mean 'unknown age', or storing an empty string "" to mean 'no email' violates Rule 3. NULL must be the designated unknown marker.
Remember	NULL != 0, NULL != "", NULL != 'N/A'. NULL is a special marker meaning unknown/inapplicable. Any comparison with NULL returns NULL (unknown), not TRUE or FALSE.

Rule 4 — Active Online Catalog Based on the Relational Model

Rule 4	Active Online Catalog (Data Dictionary)
Definition	The database schema (table names, column names, data types, constraints, views, indexes) must be stored as tables in the database itself — the system catalog. It must be queryable using the same relational language (SQL) used for regular data.
Real World	In MySQL: <code>SELECT * FROM information_schema.TABLES;</code> — you can query the catalog the same way you query your own tables. The schema is stored as relational data.
Violation	A DBMS that stores its schema in a proprietary binary file format that cannot be queried with SQL, and requires special admin tools to read — violates Rule 4.
Remember	The catalog (metadata) = tables too! You can query <code>INFORMATION_SCHEMA</code> in SQL to find all tables, columns, and constraints — same SQL, same relational model.

Rule 5 — Comprehensive Data Sub-Language Rule

Rule 5	Comprehensive Data Sub-Language Rule
Definition	The DBMS must support at least one well-defined language that covers: data definition (<code>CREATE/ALTER/DROP</code>), data manipulation (<code>SELECT/INSERT/UPDATE/DELETE</code>), integrity constraints, transaction management (<code>COMMIT/ROLLBACK</code>), and authorization (<code>GRANT/REVOKE</code>). This language must be expressible in a linear string format.
Real World	SQL covers ALL of these: DDL (<code>CREATE TABLE</code>), DML (<code>SELECT, INSERT</code>), DCL (<code>GRANT</code>), TCL (<code>COMMIT</code>) — one language handles everything through text commands.
Violation	A database that requires you to define tables using a GUI form (no SQL DDL), or one that has no way to express integrity constraints in the language — violates Rule 5.
Remember	SQL is the language Codd envisioned. One complete language must do everything: define, manipulate, secure, and control transactions.

Rule 6 — View Updating Rule

Rule 6	View Updating Rule
Definition	All views that are theoretically updatable must actually be updatable by the system. A view is theoretically updatable if there is a clear, unambiguous way to map the change back to the base tables.

Real World	A view showing active employees (WHERE Status='Active') is theoretically updatable — updating an employee's name through the view should update the base Employee table directly.
Violation	A DBMS that creates views but makes ALL views read-only (even simple single-table views with no aggregation) violates Rule 6. Most modern DBMS allow simple view updates.
Remember	Simple views on one table = should be updatable. Complex views (with JOIN, GROUP BY, DISTINCT, aggregates) may not be — that is acceptable. But simple ones MUST be.

Rule 7 — High-Level Insert, Update, and Delete Rule

Rule 7	High-Level Insert, Update & Delete
Definition	The DBMS must support set-level (high-level) insert, update, and delete operations — not just row-by-row processing. A single command should be able to modify multiple rows at once.
Real World	UPDATE Employee SET Salary = Salary * 1.10 WHERE Dept = 'IT'; — one SQL statement updates ALL IT department employees at once. This is set-based, not row-by-row.
Violation	A system that only allows you to update one row at a time (cursor-based, loop-based only) and has no set-based UPDATE/DELETE — violates Rule 7.
Remember	SQL's UPDATE, DELETE, INSERT...SELECT are set-level operations. Always prefer set-based SQL over row-by-row loops (cursors) — it is more efficient and follows Rule 7.

Rule 8 — Physical Data Independence

Rule 8	Physical Data Independence
Definition	Changes to the physical storage structure of the database (e.g., changing file layout, adding/removing indexes, moving data to a different storage device) must NOT require changes to application programs or SQL queries.
Real World	A DBA adds an index on Employee.Dept_ID to speed up queries. All existing SQL queries continue to work exactly as before — no application code changes are needed.
Violation	A DBMS where adding an index breaks existing queries, or where applications must be rewritten when the DBA changes the physical file format — violates Rule 8.
Remember	You should be able to add indexes, change tablespaces, reorganize storage — and the application code never needs to know. SQL is the abstraction layer.

Rule 9 — Logical Data Independence

Rule 9	Logical Data Independence
Definition	Changes to the logical structure (schema) of the database — such as splitting or merging tables, adding columns — must not require changes to application programs, as long as the information they need is still accessible (typically through views).
Real World	A DBA splits the Employee table into Employee_Personal and Employee_Work. A view named Employee is created that JOINS them. Existing applications using 'Employee' continue to work without modification.
Violation	A DBMS where renaming a column or splitting a table forces all application code to be rewritten — violates Rule 9. Views are the key mechanism for achieving this.
Remember	Logical independence is HARDER to achieve than physical. Views are the main tool. This is why good schema design and view abstraction are critical in enterprise systems.

Rule 10 — Integrity Independence

Rule 10	Integrity Independence
Definition	Integrity constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK constraints) must be definable in the relational data sub-language (SQL) and stored in the catalog — NOT hard-coded in application programs.
Real World	CONSTRAINT chk_age CHECK (Age >= 18) defined in the CREATE TABLE statement — enforced by the DBMS for every insert/update, regardless of which application accesses the data.
Violation	An application that manually checks 'if age < 18 then reject' in Java/Python code, but does not enforce it at DB level — violates Rule 10. Another app can bypass this check.
Remember	Constraints belong in the DATABASE, not in application code. If a constraint is only in the app, any direct DB access, ETL job, or new app can bypass it silently.

Rule 11 — Distribution Independence

Rule 11	Distribution Independence
Definition	The end user and applications should not need to know whether the database is stored on one machine or distributed across many machines/locations. Queries should work identically in both cases.

Real World	An Oracle RAC cluster distributes data across 4 servers. The developer writes <code>SELECT * FROM Orders WHERE Cust_ID=101</code> and gets results — without knowing or caring which server holds that data.
Violation	A distributed database system where SQL queries must explicitly specify which server/node to query (e.g., <code>SELECT * FROM Server2.Orders</code>) — violates Rule 11. The distribution is exposed.
Remember	Distribution should be transparent — invisible to developers and end users. This is the principle behind cloud databases and distributed SQL systems like Google Spanner.

Rule 12 — Non-Subversion Rule

Rule 12	Non-Subversion Rule
Definition	If the DBMS provides a low-level (record-at-a-time) interface, that interface must NOT be allowed to bypass integrity constraints or security rules defined at the relational (set-level) interface.
Real World	Even if a developer uses a JDBC cursor to read row-by-row, the database still enforces FOREIGN KEY constraints, CHECK constraints, and access controls. Low-level access cannot bypass these.
Violation	A DBMS that has a C API or COBOL batch interface that allows direct record updates bypassing foreign key checks — violates Rule 12. Data integrity can be silently broken.
Remember	No back door to your data. Any low-level API, cursor, or batch process must still respect ALL constraints. This is why stored procedures and APIs should enforce the same rules as SQL.

4. All 12 Rules — Quick Reference Table

Rule	Name	Core Principle	Key Concern	Category
0	Foundation Rule	Manage ALL data through relational capabilities only	System integrity	Foundation
1	Information Rule	All data stored as table values — no pointers	Data representation	Data Storage
2	Guaranteed Access	Table+PK+Column = unique access to any value	Primary keys	Data Access
3	NULL Treatment	NULL = unknown/inapplicable; not zero or blank	Missing data	Data Integrity
4	Active Online Catalog	Schema stored as tables, queryable by SQL	Metadata access	Metadata
5	Comprehensive Language	One language handles DDL+DML+DCL+TCL	SQL completeness	Language
6	View Updating	Updatable views must actually be updatable	View writability	Views
7	High-Level DML	Set-based INSERT/UPDATE/DELETE supported	Bulk operations	DML
8	Physical Independence	Storage changes don't break applications	Storage abstraction	Independence
9	Logical Independence	Schema changes don't break applications	Schema abstraction	Independence
10	Integrity Independence	Constraints live in DB catalog, not in app code	Constraint storage	Integrity
11	Distribution Independence	User unaware of data distribution	Distributed DB	Distribution
12	Non-Subversion	Low-level access cannot bypass constraints	Security bypass	Security